

Applied Machine Learning - Advanced

Prof. Daniele Bonacorsi, Dott. Luca Clissa

Lecture 3

[07.05.2026]

Data Science and Computation PhD + Master in Bioinformatics
University of Bologna

MLP for regression and MLP for classification

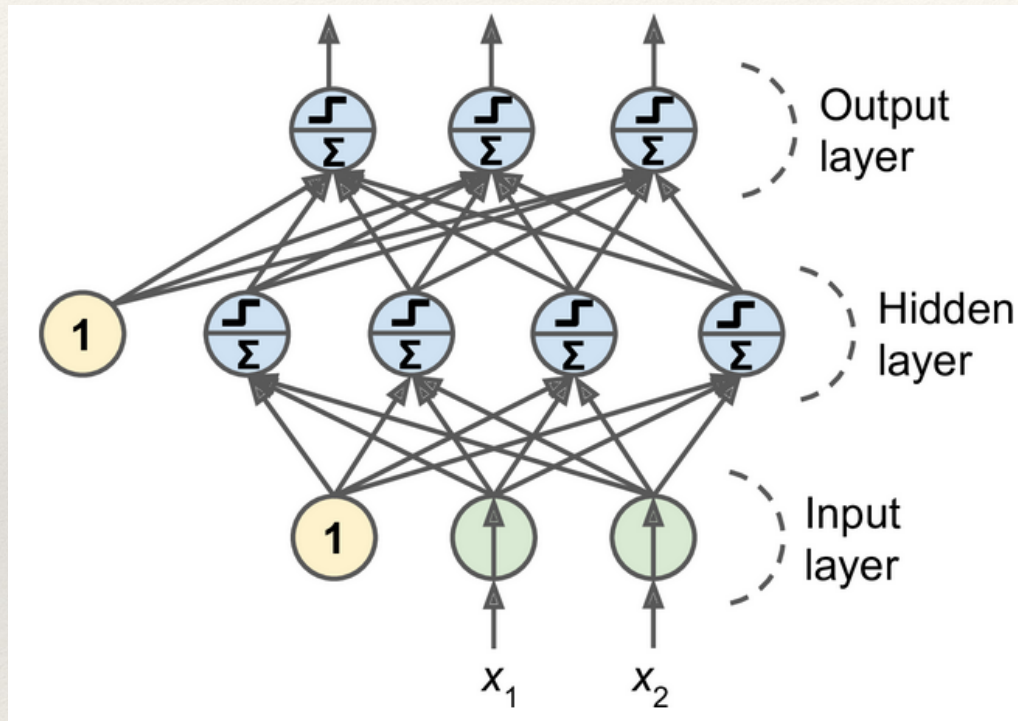
We have seen what a NN is and where its “power” lies.

So, **what** can you do with MLPs? And **how** (*) you do it?

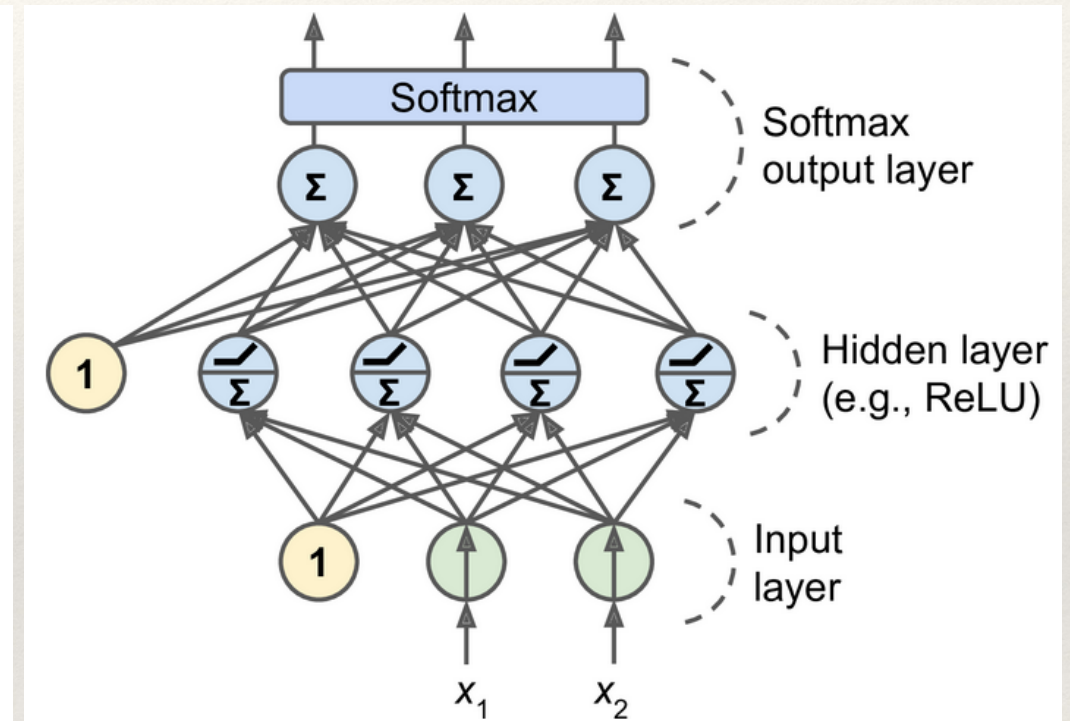
(*) CAVEAT: *VERY difficult to give you “working recipes” for all possible applied-ML situations...
but let’s try to consolidate some basic heuristics and hints*

MLP Architectures

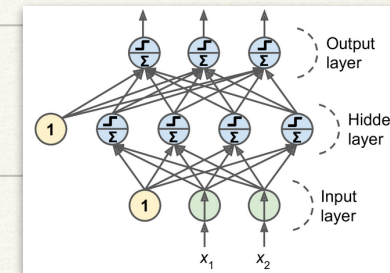
Regression MLP



Classification MLP



Regression MLP



MLPs can be used for **regression** tasks

1 vs many output neurons depending on **univariate vs multivariate regression**

- univariate regression e.g. predict a house price $\rightarrow$ need 1 output neuron
- multivariate regression e.g. predict center of a 2D object $\rightarrow$ need 2 output neuron

In general, no activation function for the output neurons, so free to output any range of values. But:

- if you want prediction to be positive, use **ReLU** in the output layer
 - ❖ or (alternatively) a smooth variant of ReLU i.e. the **softplus** ($\text{softplus}(z) = \log(1 + \exp(z))$, i.e. close to 0 when z is negative, close to z when z is positive)
- if you want prediction to fall within a given range of values, use the **logistic** function or the **hyperbolic tangent**, and scale the labels to the appropriate range (0 to 1 for the logistic function, or -1 to 1 for the hyperbolic tangent)

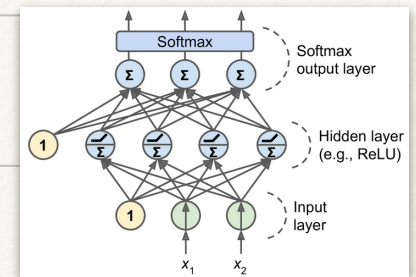
The **loss function** to use during training is typically the **MSE** ($= (1/n) * \sum (\text{predicted} - \text{true})^2$)

- but if you have many outliers in the training set, **MAE** ($= (1/n) * \sum |\text{predicted} - \text{true}|$), works better
 - ❖ or use the **Huber** loss, which is a combination of both (quadratic when the error is smaller than a threshold δ - typically 1, but linear when the error is larger than δ). This makes it less sensitive to outliers than MSE, and it is often more precise and converges faster than MAE

Regression MLP: architecture

Hyperparameter	typical value
# input neurons	One per input feature (e.g. $28 \times 28 = 784$ for MNIST)
# hidden layers	Depends on the problem. Typically 1 to 5.
# neurons per hidden layer	Depends on the problem. Typically 10 to 100.
# output neurons	1 per prediction dimension
Hidden activation	ReLU (or SELU)
Output activation	None or ReLU/Softplus (if positive outputs) or Logistic/Tanh (if bounded outputs)
Loss function	MSE or MAE/Huber (if outliers)

Classification MLP



MLPs can be used for **classification** tasks

1 vs many output neurons depending on **single-label vs multi-label binary classification**, all using the **logistic activation function**

- dedicate one output neuron for each positive class: the output(s) will be a number between 0 and 1, which you can interpret as the estimated probability P of the positive class, with the estimated probability of the negative class being $1-P$
 - ❖ single-label binary classification: e.g. spam/not-spam
 - ❖ multi-label binary classification: e.g. spam/not-spam together with urgent/not-urgent
 - ❖ note that classes might be non-exclusive: in this case, probabilities will not add up to 1

1 output neuron per class in case of **multiclass classification**, using the **softmax** activation function for the whole output layer

- you need to have one output neuron per class, and you should use the softmax activation function to ensure that all the estimated probabilities are between 0 and 1 and that they add up to 1 (required if the classes are exclusive)
 - ❖ e.g. classes 0 through 9 for MNIST classification: each instance can belong only to a single class, there are several, and they are exclusive

The **loss function**: since we are predicting probability distributions, the **cross-entropy** (aka **log loss**) is generally a good choice

Classification MLP: architecture

Hyperpar.	single-label binary classif.	multi-label binary classif.	multiclass classif.
Input and hidden layers	<i>Same as regression</i>		
# output neurons	1	1 per label	1 per class
Output layer activation	Logistic	Logistic	Softmax
Loss function	Cross-Entropy	Cross-Entropy	Cross-Entropy

More on: Training a NN

Training an ANN

Let's focus again (and more) on this question:

- How do you compute the weights. I.e. **how do you train a NN?**

NNs are a pretty old class of algorithms that have been originally inspired by one goal: **build a machine that can mimic the brain.**

We need to get a sense of how this basic logic drove their conceptual development, though, what we can expect them to ultimately do.

[] CAVEAT: not intended to be a neuroscience-compliant explanation!*

The origins of NNs is as algorithms that try to mimic the brain.

Think about the senses. The brain can learn to process images (= see), process acoustic waves (= hear), process other signals (e.g. sense of touch)... quite complex! and different tasks! and you are not even close to the end of the list of what brains can do (e.g. think?)

If you mimic by “coding”, it is - if you want to mimic the brain - as if you need to write lots of different pieces of software... or, you just make a hypothesis. You speculate that the way the brain manages to do all of these different things is not worth like a thousand different software programs, but instead encoding **a single learning algorithm** [*].

Brain = one single learning algorithm? A divagation..

The “**one single learning algorithm**” hypothesis is currently just an hypothesis. But it is not with 0 evidence.

E.g. auditory cortex. You hear voices because your ears mechanically get a vibration and route this signal to the auditory cortex that process it as a sound signal. Same for visual, or somatosensory brain areas.

Neuroscientists have found out that if one “cuts the wire” from the ears to the auditory cortex, the brain re-wires (“neuro-rewiring experiments”): e.g. the signal from the eyes to the optic nerve eventually gets routed to the auditory cortex. If you do it turns out that the auditory cortex somehow “learns to see”. To a surprisingly large extent, it seems somehow as if we can plug in almost any sensor to almost any part of the brain and the brain will learn to deal with it [*].

[] CAVEAT: not intended to be a neuroscience-compliant explanation!*

So, if the same piece of physical brain tissue may be able to process sight or sound or touch, then maybe there is just one learning algorithm behind, that can process any of them.

So - if the former is true - in trying to emulate the brain, instead of implementing a thousand different algorithms, maybe what we need is to figure out **some approximation of whatever the brain's one-and-only-one learning algorithm is, and implement it.**

This might be our best shot at making real progress towards the AI dream of building someday truly intelligent machines.

But note: **the NN logical/AI-goal motivation behind is not the reason why we do use them; we do use them because they work!**

- but it is nice to keep a window open to what this might imply..

Training a **MLP**: the **BP** algo

For many years, researchers struggled to find a way to train MLPs.

1986, Rumelhart/Hinton/Williams introduced the **back-propagation (BP) training algorithm** [\[Ref-RumelhartHintonWilliams\]](#) - still used today

- it is simply GD using an efficient technique for computing the gradients automatically
- call “BP” because it does two **propagations** through the network (one forward, one **backward**), and is able to compute the gradient (*) of the network’s error wrt every single model parameter. I.e. it can find out how each connection weight and each bias term should be tweaked in order to reduce the network error
 - ❖ Automatically computing gradients is called **automatic differentiation (autodiff)**. Various techniques exist, BP uses **reverse-mode autodiff**. It is fast and precise, and is well suited when the function to differentiate has many variables (e.g. connection weights) and few outputs (e.g. one loss)
- once it has these gradients, it just performs a regular GD step
- the whole process is iteratively repeated - in **epochs** - until the network converges

So... how do **GD** and **BP** work together?!

Which is the difference between GD and BP algorithms, **in simple terms**?

Think of the hiker example.

GD: the big hill represents the error or loss function of a problem. The goal is to find the lowest point on the hill (*which corresponds to the minimum error*). GD is like the hiker following a path towards the lowest point on the hill, by taking small steps in the direction of the steepest slope, and hence he gets down-hill... eventually the goal would be to reach the bottom of the hill (*which represents the minimum error*). One path, executed just once, might work but might not be the optimal, though.

BP: (in a NN only) it is a way for the N to figure out which direction the hiker had better go, ultimately (*the NN needs to know how each weight and bias in the NN contributes to the error*). It's like a trail of breadcrumbs left by the hiker on its way, moved and "tuned" again in the reverse direction now, going back up-hill. The hiker starts at the bottom of the hill and leaves breadcrumbs along the way as it climbs up (they represent the gradients, which tell the NN how much each weight and bias needs to change to decrease the error). By following the adjusted breadcrumbs, at the next GD step, the hiker can follow a better path (*i.e. the NN can adjust its weights and biases using GD to eventually reach the minimum error in next rounds*).

So... how do **GD** and **BP** work together?!

In brief:

GD is the process of taking small steps to find the lowest point (i.e. an optimization algorithm that, in a NN, iteratively updates weights/biases based on the computed gradients),

while

BP is the process of figuring out how each step (change in weights/biases) should be in order to reach the actual lowest point (i.e. the algorithm used to calculate the gradients of the error function with respect to weights/biases, using the chain rules of calculus, enabling GD to perform the weight updates).

In training a NN, **they are both needed.**

Training a **MLP**: the **GD+BP** algo (in one sentence)

GD+BP algo in one sentence:

For each training instance, the algo first makes a prediction, measures the error and preserve info on all steps (BP in NN, here), then goes through each layer in reverse to measure the error contribution from each connection (**backwards**/reverse pass, BP), and finally slightly tweaks the connection weights to reduce the error (**forward** pass, GD).

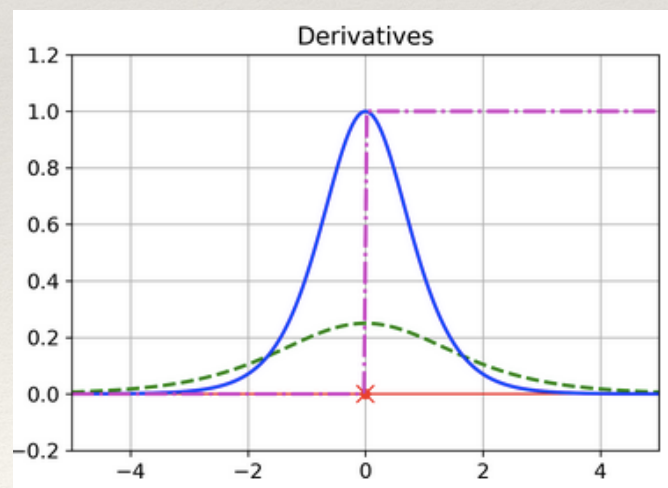
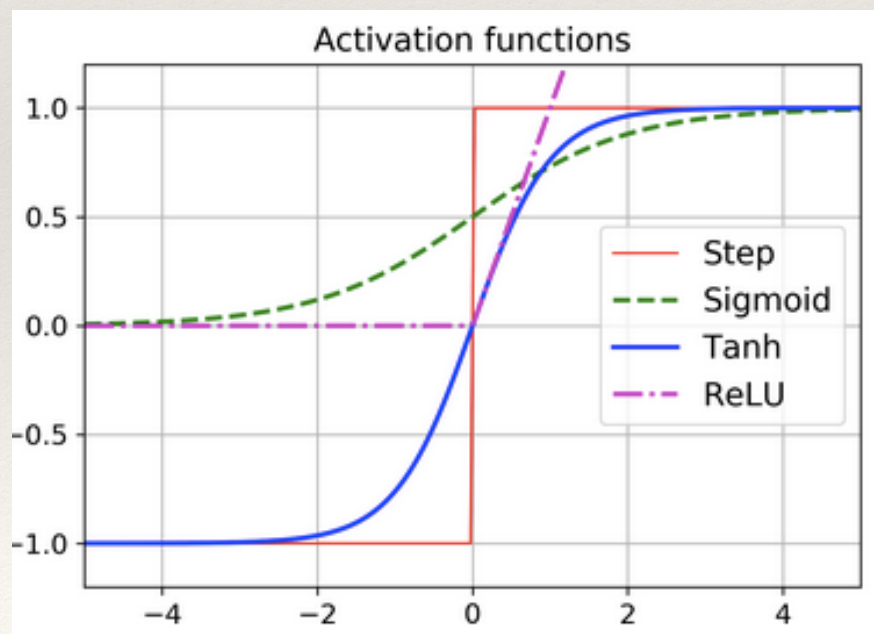
Training a **MLP**: the **GD+BP** algo (more details)

- it divides the training set in mini batches. It handles one mini-batch at a time (with e.g. 32 instances each). It goes through the full training set multiple times. Each pass on all of it is called an **epoch**
- each mini-batch is passed to the network's input layer, which just sends it to all units of the first hidden layer
- for every instance in this minibatch, BP computes the output of all the neurons in this layer
- the result is passed on to the next layer, its output is computed and passed to the next layer, and so on until we get the output of the output layer
- **This is the forward pass**: it is exactly like making predictions, except all intermediate results are preserved since they are needed for the backward pass.
- BP measures the **network's output error** (i.e. using a loss function that compares the desired output and the actual output of the network, and returns some measure of the error)
- BP computes how much each **output** (layer) connection contributed to the error. This is done analytically by applying the (calculus) chain rule (which makes this step fast and precise)
- BP then measures how much of these error contributions came from each connection in the layer below, again using the chain rule - and repeat this until the algorithm reaches the input layer.
- **This is the backwards pass**. This reverse pass efficiently measures the error gradient across all the connection weights in the network by propagating the error gradient backward through the network
- Finally, BP performs a GD step to tweak all the connection weights in the network, using the error gradients it just computed

BP and activation functions

Choice of the activation functions

- the BP authors made a key change to the MLP architecture: they replaced the step function with the **logistic** function, $\sigma(z) = 1 / (1 + \exp(-z))$. BP uses GD which can thus make a progress at every step
 - Essential to avoid having only flat segment (no gradients, i.e. GD unable to move on a flat surface)
- Other functions would have work well too: **hyperbolic tangent** function $\tanh(z) = 2\sigma(2z) - 1$, **REctified Linear Unit** function: $\text{ReLU}(z) = \max(0, z)$



BP and **initialisation**

Initialisation matters.

It is important to **initialise all the hidden layers' connection weights randomly**, or else training will fail

- e.g. if all weights and biases stay initialised to 0, then all neurons in a given layer will be identical, thus BP will affect them in the same way, and they will remain identical → no actual training!
- you need to initialise, so you break the symmetry, and allow BP to train a diverse team of neurons

NNs' building blocks

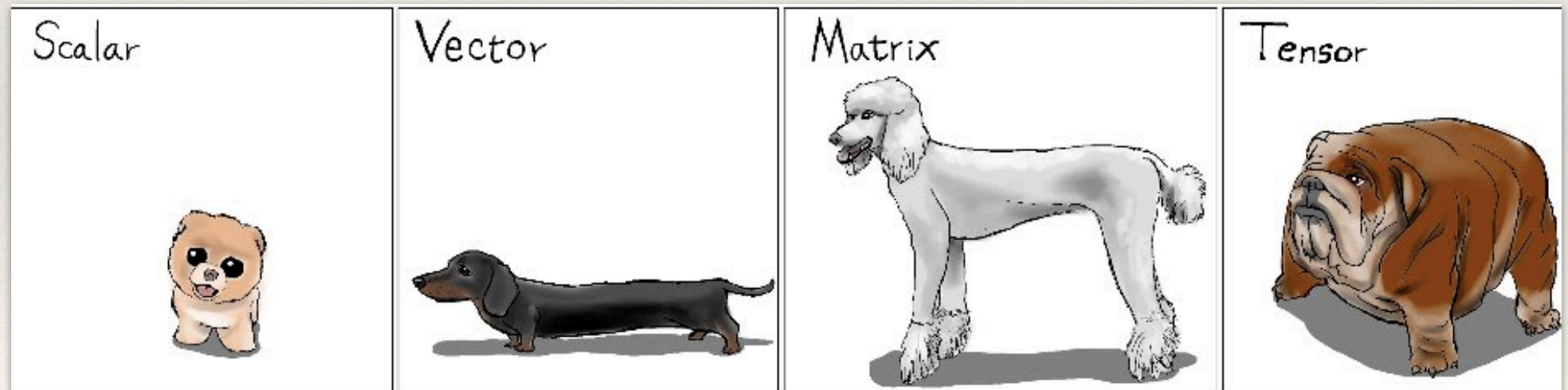
Tensors and tensor operations

Part 1

Tools.. on the data!

Let's "match" what we introduced about the **tools** we will use, with some notes on the actual **data structures** we will use for our datasets.

Data tensors



Real-world example of data tensors

Let's make data tensors more concrete, with some examples of what you may encounter in real life (or: "you might need to force your dataset into").

The data you'll manipulate will almost always fall into one of the following categories:

Vector data: **2D tensors** of shape (samples, features)

Timeseries data (or "sequence data"): **3D tensors** of shape (samples, timesteps, features)

Images : **4D tensors** of shape (samples, height, width, channels) or (samples, channels, height, width)

Videos: **5D tensors** of shape (samples, frames, height, width, channels) or (samples, frames, channels, height, width)

Let's discuss each.

Vector data

The most common case. In such a dataset:

- every single data point can be encoded as a vector
- → a batch of data will be encoded as a **2D vector data tensor** (i.e. an array of vectors)
- the first axis is the **samples axis** and the second axis is the **features axis**

Vector data

E.g.:

A dataset of people, where we consider e.g. each person's age, ZIP code, and income

- Each person can be characterised as a vector of 3 values, and thus an entire dataset of 100k people can be stored in a 2D tensor of shape (100'000, 3).

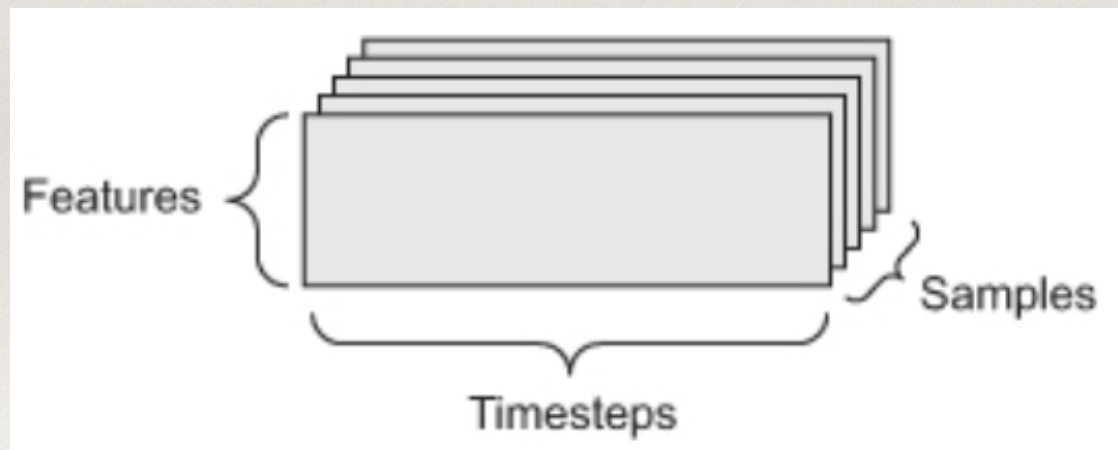
A dataset of text documents, where we represent each document by the counts of how many times each word appears in it (out of a dictionary of e.g. 20k common words)

- each document can be encoded as a vector of 20k values (one count per word in the dictionary), and thus an entire dataset of 500 documents can be stored in a tensor of shape (500, 20'000)

Time-series (or sequence data)

Whenever time matters in your data (or the notion of sequence order), it makes sense to store it → a **3D time-series (or sequence) data tensor** with an explicit time axis

- every sample can be encoded as a sequence of vectors (i.e. a 2D tensor)
- → a batch of data will be encoded as a **3D tensor** which has thus an explicit **time axis**
- the time axis is always the second axis (axis of index 1), by convention



Time-series (or sequence data)

E.g.:

A dataset of stock prices. Every minute, we store e.g. the current price of the stock, the highest price in the past minute, and the lowest price in the past minute.

- → every minute's information is encoded as a (3D) vector → an entire day of trading (390 minutes) is encoded as a 2D tensor of shape $(390, 3)$ → 252 trading days' worth of data can be stored in a 3D tensor of shape $(252, 390, 3)$. Here, each sample would be one day's worth of data

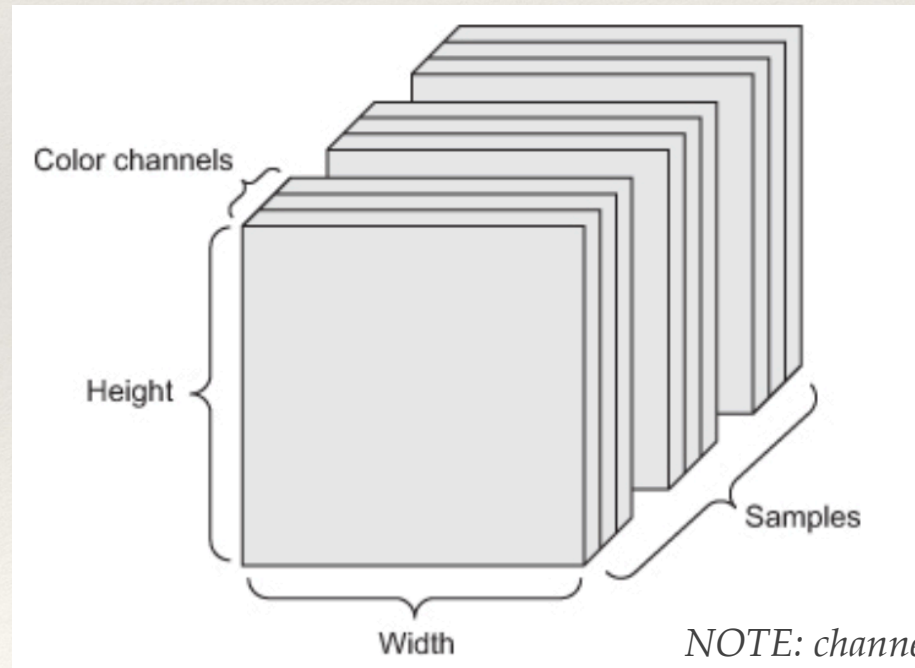
A dataset of tweets. We encode each tweet as a sequence of (now) 280 characters out of an alphabet of 128 unique characters

- → each character can be "one-hot-encoded" as a binary vector of size 128 (an all-0s vector except for a 1 entry at the index corresponding to the character) → each tweet can be encoded as a 2D tensor of shape $(280, 128)$, and a dataset of 1M tweets can be stored in a tensor of shape $(1000000, 280, 128)$

Image data

Images typically have 3 dimensions: height, width, and color depth. This is for each sample, so → **4D image data tensors**

- grayscale images (e.g. MNIST) have only a single color channel and could thus be stored in 3D tensors, by convention image tensors are always 4D, with a one-dimensional color channel for grayscale images
- → a batch of 128 grayscale images of size 256×256 could thus be stored in a tensor of shape $(128, 1, 256, 256)$, while a batch of 128 color images could be stored in a tensor of shape $(128, 3, 256, 256)$
 - ❖ NOTE: channels-last (here) vs channels-first conventions → see next slide



NOTE: channels-first convention

Image data

Careful about the conventions, for image data tensors.

There are 2 conventions for their shapes:

- the channels-last convention (e.g. used by TensorFlow)
 - ❖ it places the color-depth axis at the end: (samples, height, width, color_depth)
- the channels-first convention (e.g. used by Theano)
 - ❖ it places the color depth axis right after the batch axis: (samples, color_depth, height, width).
- And the Keras framework? Needless to say, it provides support for both formats.

Video data

A video can be interpreted as a sequence of frames, each frame being a color image → a vector of 4D image data tensors → **5D video data tensors**

- Video data is one of the few types of real-world data for which you'll need 5D tensors..
- → each frame can be stored in a 3D tensor (height, width, color_depth)
- → a sequence of frames can be stored in a 4D tensor (frames, height, width, color_depth)
- → a batch of different videos can be stored in a 5D tensor of shape (samples, frames, height, width, color_depth)

Video data

E.g.

A dataset of YouTube videos. A 60-second, 144×256 YT video clip (w/ colour) sampled at 4 frames per second would have 240 frames. A batch of 4 such video clips would be stored in a tensor of shape $(4, 240, 144, 256, 3)$.

- total: 106,168,320 values (!)
- assume the *dtype* of the tensor was float32 → each value would be stored in 32 bits → the tensor would be 405 MB.
 - ❖ This builds us quite heavy storage needs for videos, in theory. In practice: videos you encounter in real life are much lighter, because they aren't stored in float32, they're typically compressed by a large factor (e.g. in the MPEG format)

NNs' building blocks

Tensors and tensor operations

Part 2

Tensor operations



Coding material

..._HandsOn_Tensors.ipynb

→ we will proceed on slides though

The gears inside NNs: tensor operations

Any computer program does what it does, if reduced to its core actions, via a combination of a relatively small set of binary operations on binary inputs

- i.e. take bits and perform combinations of ANDs, ORs, NORs, ..

Much along the same lines, all “*transformations*” that a deep NN can learn can be reduced to a handful of **tensor operations** applied to tensors of numerical data

- i.e. take tensors of various ranks, and do additions, multiplications, ..

Inside a layer instance: basic tensor ops

```
my_network = keras.models.Sequential([
    keras.layers.Dense(512, activation='relu'),
    keras.layers.Dense(10, activation='softmax')
])
```


$$\text{output} = \text{relu}(\text{dot}(W, \text{input}) + b)$$

Take our first NN (see notebook). We built the NN by stacking Dense layers on top of each other. Take one (see above):

- this layer can be interpreted as a **function**: it takes as “input” a tensor and returns as “output” a new **representation** of the input tensor

Specifically, the function does:

- a **dot** product, an addition and then applies a function **relu(x)** (which is basically **max(x,0)**)

(NOTE: of course all this is linear algebra.. here: using Numpy snippets instead)

Element-wise operations

If we naively implement `relu(x)` and the addition in Python using Numpy, we may just use `for` loops:

```
def naive_relu(x):  
    assert len(x.shape) == 2  
  
    x = x.copy()  
  
    for i in range(x.shape[0]):  
        for j in range(x.shape[1]):  
            x[i, j] = max(x[i, j], 0)  
    return x
```

```
def naive_add(x, y):  
    assert len(x.shape) == 2  
  
    assert x.shape == y.shape  
    x = x.copy()  
  
    for i in range(x.shape[0]):  
        for j in range(x.shape[1]):  
            x[i, j] += y[i, j]  
    return x
```

The `relu` operation and the addition are *element-wise* operations

- ops applied independently to each entry in the tensors being considered
- highly adequate to massively parallel implementations (i.e. *vectorized* implementations)

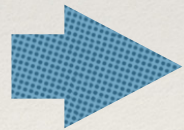
Same principle applies for element-wise multiplication, subtraction, ..

Element-wise operations: behind the scenes

What's happening behind the scene?

- we use Numpy arrays → ops available as well-optimized built-in Numpy functions → they delegate the hard work to BLAS (Basic Linear Algebra Subprograms) implementations, that are low-level, highly parallel, efficient tensor-manipulation routines
 - ❖ typically implemented in Fortran or C

Try it out on the notebook.



Also for larger tensors..
It is pretty fast.

```
[25] x = np.array([[1, 1],  
                  [1, 1]])  
      y = np.array([[2, 2],  
                  [2, 2]])
```

```
[28] z = x + y  
      z  
  
      array([[3, 3],  
            [3, 3]])
```

```
[27] %%time  
      z = x + y  
      z
```

```
CPU times: user 20 µs, sys: 3 µs, total: 23 µs  
Wall time: 27.7 µs
```


Element-wise operations: broadcasting

But we have “naive” implementations

- i.e. only supporting the addition of 2D tensors with identical shapes

Already in the Dense layer we have an addition of a 2D tensor with a vector. What happens?

Broadcasting. When possible, and if there's no ambiguity, the smaller tensor will be broadcasted to match the shape of the larger tensor, in 2 steps:

- axes (called *broadcast axes*) are added to the smaller tensor to match the `ndim` of the larger tensor
- the smaller tensor is repeated alongside these new axes to match the full shape of the larger tensor

See on the notebook



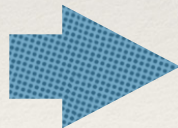
```
[ ] def naive_add_matrix_and_vector(x, y):  
    assert len(x.shape) == 2  
    assert len(y.shape) == 1  
    assert x.shape[1] == y.shape[0]  
    x = x.copy()  
  
    for i in range(x.shape[0]):  
        for j in range(x.shape[1]):  
            x[i, j] += y[j]  
    return x
```


Dot products

The **tensor product** (or **dot product**) is the most common and useful tensor op

- NOTE: not to be confused with an element-wise product!
 - ❖ in Numpy/Keras/TF/Theano: use the `*` operator
- contrary to element-wise operations, a dot product combines entries in the input tensors
 - ❖ in Numpy/Keras/Theano (not in TF): use the `dot` operator

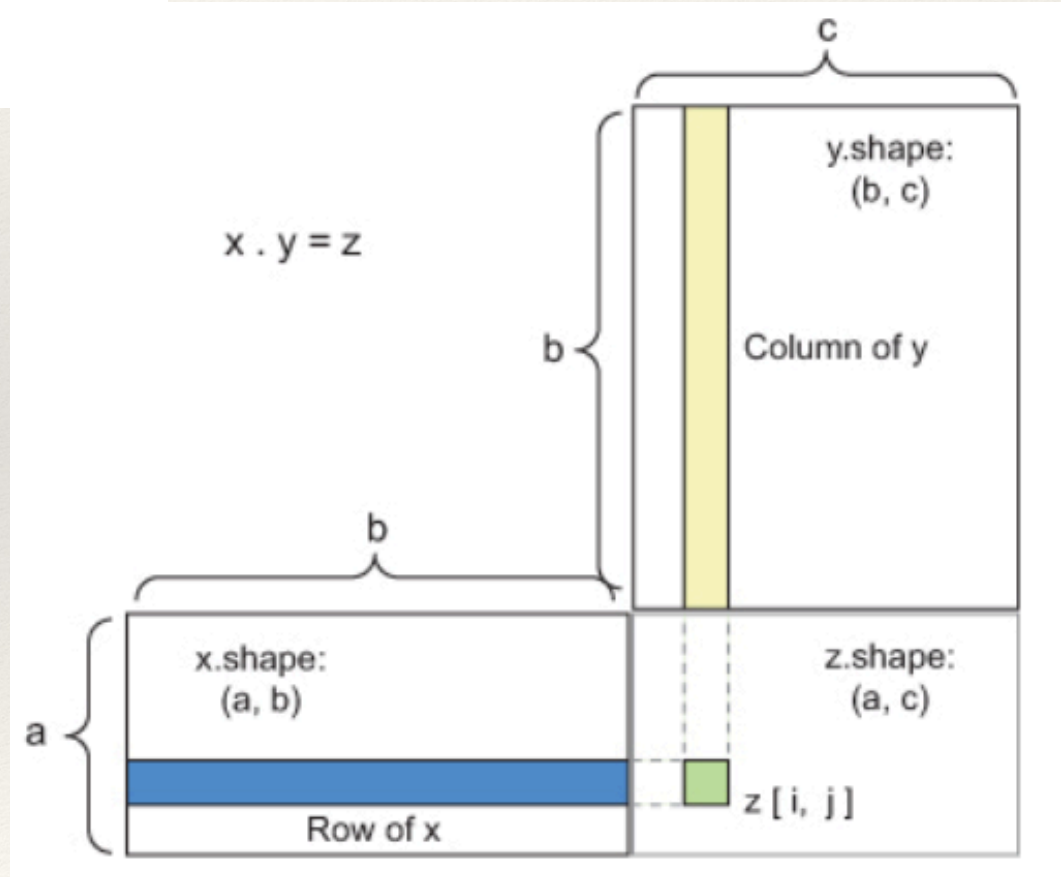
See on the notebook.



Dot products “visualised”

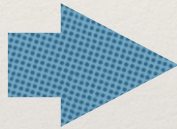
$$\begin{bmatrix} A & B \\ C & D \\ E & F \end{bmatrix}_{a \times b} \times \begin{bmatrix} G \\ H \end{bmatrix}_{b \times c} = \begin{bmatrix} A \times G + B \times H \\ C \times G + D \times H \\ E \times G + F \times H \end{bmatrix}_{a \times c}$$

Visualising tensors as
“boxes” of numbers, and
aligning them



Tensor reshaping (incl. transposition)

See on the notebook.



From a geometric interpretation of tensor ops..

The tensors' content can be interpreted as point coordinates in some geometric space $\rightarrow$ tensor operations have a geometric interpretation

- e.g. a vector $A[1,2]$ can be interpreted as the coordinate of a point in a 2D space, and one can even draw a vector as if it was a position vector of a physical point w.r.t the origin of a coordinate system
- if you take another vector and make the sum, analytically or visually, you get the coordinate of a third point in the same geometrical space

This is general - elementary geometric operations such as affine transformations, rotations, scaling, .. can be expressed as tensor operations

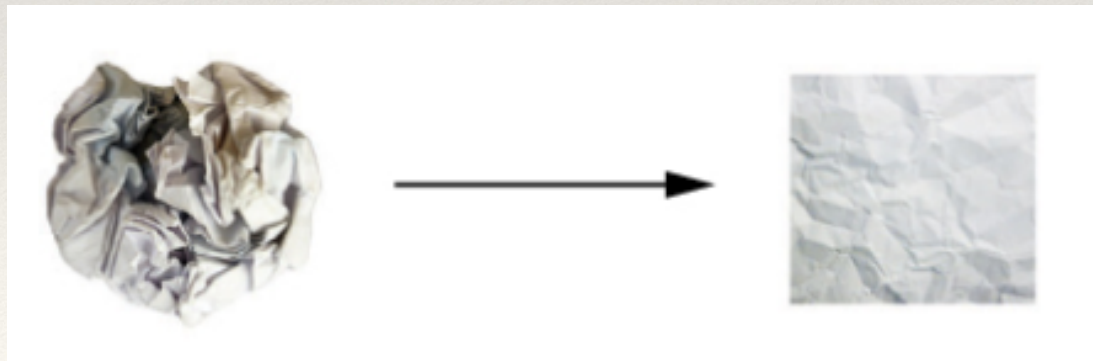
- e.g. a rotation of a 2D vector by an angle θ can be achieved via a dot product with a 2×2 rotation matrix $R=[u,v]$, where u and v are vector of the plane, namely $u=[\cos(\theta), \sin(\theta)]$ and $v=[-\sin(\theta), \cos(\theta)]$

.. to a geometric interpretation of DL

NNs consist entirely of chains of tensor ops → all these tensor ops are just geometric transformations of the input data → we can interpret a NN as a (very complex) **geometric** transformation in a high-dimensional space, implemented via a long series of simple steps (operations)

Visual example:

- DL as the process of uncrumpling a complicated manifold of data (a paper ball done by sheets of different colours), i.e. figure out which set of many (deep!) transformations - need to be applied one after another (layers!) that may make the original paper sheets cleanly separable again



Gradient-based optimization



Trainable parameters

Let's restart from here:

```
my_network = keras.models.Sequential([  
    keras.layers.Dense(512, activation='relu'),  
    keras.layers.Dense(10, activation='softmax')  
])
```


$$\text{output} = \text{relu}(\text{dot}(W, \text{input}) + b)$$

W and b are **trainable parameters** (or **weights**) of that layer, and they are **attributes** of the layer (namely, the **kernel** and **bias** attributes). They contain the information learned by the NN from exposure to training data

- from the Basic part of the course: the θ parameters..

They are **tensors**. What's in them?

- initially, they are filled with small random values (**random initialization**)
- of course, applying a function to random stuff does not yield any useful representation - it is just a starting point. Weights need to be adjusted, based on some feedback. This gradual adjustment is indeed what we call the **training** (namely, the "learning" part in "machine learning")

❖ see next

Training

The training happen in loops. Let's zoom into one loop:

1. Draw a **batch** of training samples x and corresponding targets y
2. Run the NN, i.e. run a **forward pass** on x to obtain predictions y_{pred}
3. Compute the loss of the NN on this batch (i.e. a measure of the mismatch between y_{pred} and y)
4. Update all weights of the network in a way that slightly reduces the loss on this batch

When you eventually end up with a NN that has a very low loss on its training data, and you call it done, you judge that the NN has "learned" to map its inputs to correct targets.

Let's revisit steps 1-4 on the basis of what happens behind the scene, and how complex it is.

Training loop evaluation

One training loop:

1. Draw a **batch** of training samples x and corresponding targets y
 - easy - just I/O code
2. Run the NN, i.e. run a **forward pass** on x to obtain predictions y_{pred}
3. Compute the loss of the NN on this batch (i.e. a measure of the mismatch between y_{pred} and y)
 - not as easy as 1, but both 2 and 3 are tensors ops - one can implement everything from scratch or use existing (optimized, sophisticated) libraries
4. Update all weights of the network in a way that slightly reduces the loss on this batch
 - this is tough. You need to update the NN's weights, decide if to increase or decrease, and by how much. How?

How to perform weights updates?

I can naively think of freezing all weights in the NN, except the one scalar coefficient being considered → then, try different values for that coefficient

- e.g. assume that the initial value (of one single coefficient) is 0.3. I do a first fwd pass on a batch of data → loss of the NN on that batch is 0.9. If I change the coefficient to 0.35 (0.25) and rerun the fwd pass, the loss increases (decreases) to 1.0 (0.8) → you update it e.g. by -0.05 to contribute to minimising the loss. And you need to repeat this for all coefficients in the NN
- terribly inefficient → 2 (expensive) fwd passes needed per each coefficient (and they are many)

A much better approach is to take advantage of the fact that all ops used in the NN are **differentiable**

- hence compute the **gradient** of the loss with regard to the NN's coefficients → you can then move the coefficients in the opposite direction from the gradient, thus decreasing the loss

Gradient as the derivative of a tensor operation

- i.e. the generalisation of the concept of derivatives to functions of multidimensional inputs (i.e. that take tensors as inputs)

Normal equation vs GD

We discussed it in the Basic part of the course in quite some detail

- we know already that an iterative method is highly desirable!

The strategy is simple:

- we modify the parameters little by little based on the current loss value on a random batch of data
- as we are dealing with a differentiable function, we can compute its gradient
- the gradient gives us an efficient way to implement the difficult **step 4** (see previous slides) as per our training loop → **we just need to update the weights in the opposite direction from the gradient, and the loss will be a little less every time**

Let's hence modify our initial training loop

- see next

SGD

The “new” training loop:

1. Draw a **batch** of training samples x and corresponding targets y
2. Run the NN, i.e. run a **forward pass** on x to obtain predictions y_{pred}
3. Compute the loss of the NN on this batch (i.e. a measure of the mismatch between y_{pred} and y)
4. Compute the gradient of the loss with regard to the network's parameters (a **backward pass**).
5. Move the parameters a little in the opposite direction from the gradient thus reducing bit a the loss on the batch

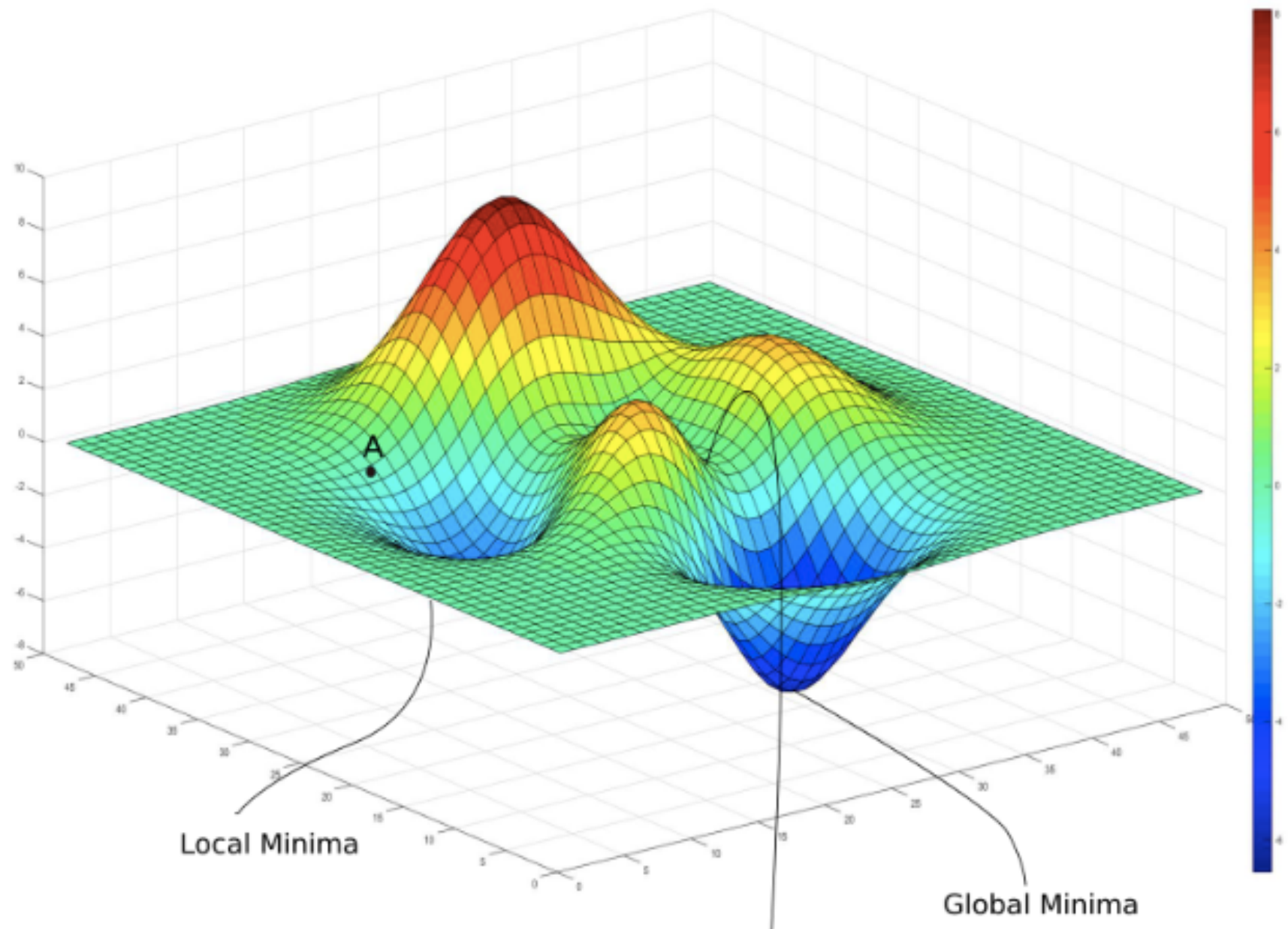
This is **mini-batch stochastic gradient descent (mini-batch SGD)**

- “stochastic” due to the random draw of samples to form each batch

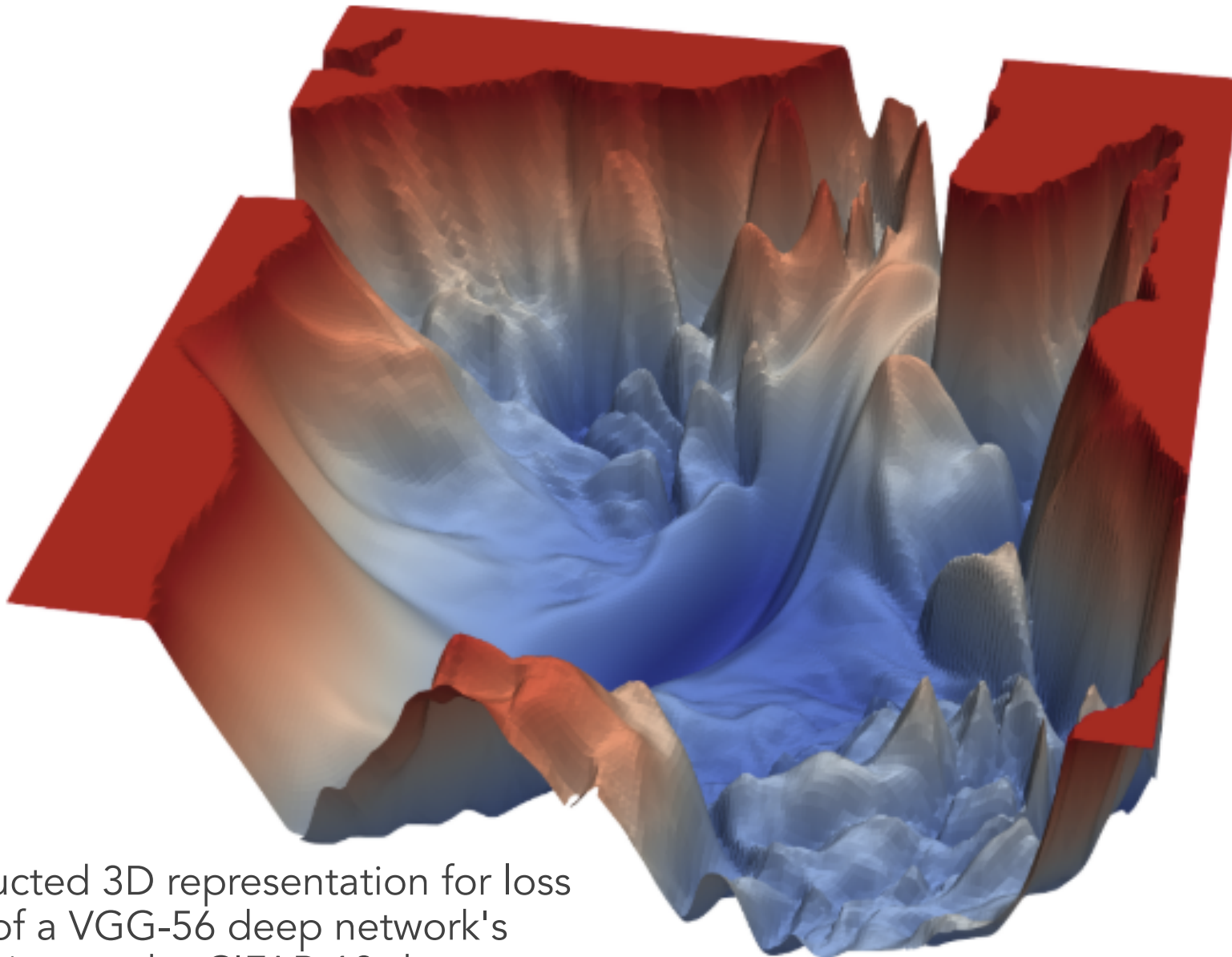
A good compromise among two extremes:

- “true SGD”: no batch $\rightarrow$ perform training loop on every training instance
- “batch SGD”: one single batch $\rightarrow$ perform training loop on all training set every time

Complexity of a loss curve? low here...



Complexity of a loss landscape? **high** here...



A constructed 3D representation for loss contour of a VGG-56 deep network's loss function on the CIFAR-10 dataset.

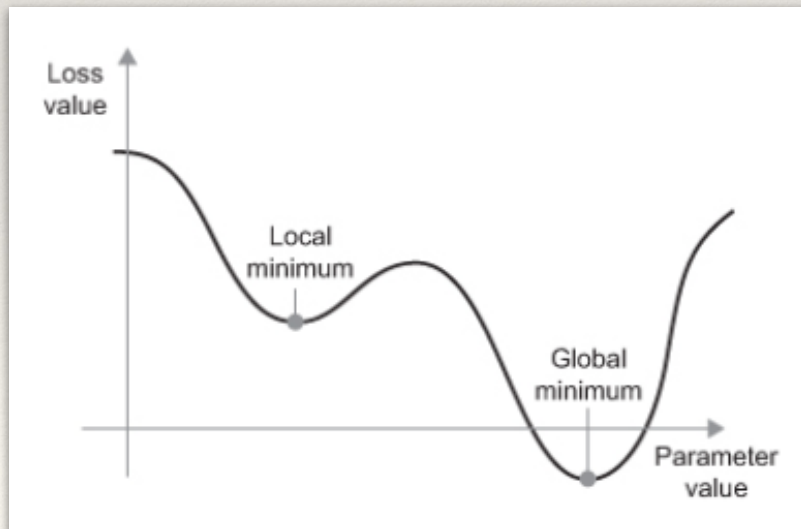
Optimisers

Multiple variants of SGD exist, and they are known as **optimisers** (or **optimisation methods**)

- e.g. **SGD with momentum**, as well as **Adagrad**, **RMSProp**, and others
- they differ by taking into account previous weight updates when computing the next weight update, rather than just looking at the current value of the gradients

The concept of momentum (derived from physics) and its application here is very interesting

- it addresses two SGD issues: convergence speed and local minima
- momentum is implemented by moving at each step based not only on the current slope value (current acceleration) but also on the current velocity (resulting from past acceleration)



```
past_velocity = 0.  
momentum = 0.1  
while loss > 0.01:  
    w, loss, gradient = get_current_parameters()  
    velocity = past_velocity * momentum - learning_rate * gradient  
    w = w + momentum * velocity - learning_rate * gradient  
    past_velocity = velocity  
    update_parameter(w)
```

(naive implementation of SGD with momentum)

Chaining derivatives: the **backpropagation** algo

In the training loop discussed so far, we can casually think of a “NN function” that is differentiable → we can compute its derivatives

In practice, a NN function consists of many tensor operations chained together, each of which has a simple, known derivative

- e.g. $f(w1, w2, w3) = a(w1, b(w2, c(w3)))$ is a NN function f composed by 3 tensor ops a , b and c acting on weight tensors $w1$, $w2$ and $w3$
- to derive f above, I would apply the chain rule ($f'(g(x)) = f'(g(x)) * g'(x)$)

Applying the chain rule to the computation of the gradient values of a NN gives rise to the **backpropagation** algo (or **reverse-mode differentiation**).

- it starts with the final loss value and works backward from the top layers to the bottom layers, applying the chain rule to compute the contribution that each parameter had in the loss value.

A note

Nowadays, and in the foreseeable future, ML researchers will implement NNs in modern frameworks that are capable of ***symbolic differentiation*** (such as TensorFlow)

- given a chain of operations with a known derivative, they can compute a gradient *function* for the chain (by applying the chain rule) that maps network parameter values to gradient values
- when you have access to such a function, the backward pass is reduced to a call to this gradient function - no need to re-implement the backpropagation algo by hand

No further deep dive on backprop in this Applied ML course, as most important is just a good intuition of how gradient-based optimization works.